

Indian Institute of Technology Delhi

COP290: Design Practices in Computer Science

Assignment 4 Report

DataFrameLib Implementation

Student Name	Megh Bhavesh Jesalpura
Entry Number	2024CS10844
Course Code	COP290
Assignment	A4
Submission Date	April 27, 2026
Github Link	https://github.com/MeghJesalpura/cop290_a4_private

Contents

1	EagerDataFrame and Exactly How Different Strategies Have Been Implemented	1
1.1	Overview of Eager Execution Philosophy	1
1.2	Input/Output and Construction Strategy	1
1.3	Strategy for Projection, Filtering, and Derived Columns	1
1.4	Strategy for Head and Sorting	2
1.5	Join Strategy in Depth	2
1.6	GroupBy and Aggregation Strategy in Depth	3
1.7	Complexity Summary for Eager Operations	3
2	LazyDataFrame and Exactly How I Am Making a Graph Using std::variant	4
2.1	Why std::variant Was Chosen	4
2.2	Plan Node Data Model	4
2.3	How Lazy Operations Build the DAG	4
2.4	From DAG to Execution: Variant-Visitor Runtime	5
2.5	Exactly How Graph Rendering Is Implemented	5
2.6	Correctness Intuition for Lazy-to-Eager Mapping	6
3	QueryOptimizer and Correctness of the Transformations That I Have Made	6
3.1	Optimizer Architecture	6
3.2	Transformation A: Constant Folding and Algebraic Simplification	6
3.3	Transformation B: Predicate Pushdown	7
3.4	Transformation C: Limit Pushdown	7
3.5	Transformation D: Projection Pushdown	7
3.6	End-to-End Correctness Statement	8
3.7	Performance Impact Summary	8
4	Design Decisions	8
5	Conclusions	9

1 EagerDataFrame and Exactly How Different Strategies Have Been Implemented

1.1 Overview of Eager Execution Philosophy

The eager engine is built around a direct and deterministic model: each API call executes immediately on an in-memory Arrow table and returns a fresh **EagerDataFrame**. This has two practical consequences in the implementation. First, there is no deferred state to maintain; each object is always materialized and fully usable. Second, operation-level errors (missing columns, invalid predicate types, unsupported join kinds) are surfaced at the same call site instead of at a later `collect()` stage.

The class stores only one private member, `std::shared_ptr<arrow::Table> table_`, and every method reconstructs an output table explicitly from Arrow arrays and schema fields. This keeps ownership simple and makes memory behavior easier to reason about.

1.2 Input/Output and Construction Strategy

CSV and Parquet loading. `read_csv()` opens a readable file handle and creates Arrow's CSV table reader with default read/parse/convert options. The resulting Arrow table is returned immediately after status checks. `read_parquet()` opens a Parquet file using `parquet::arrow::OpenFile`, reads the entire table, and wraps it in **EagerDataFrame**.

From-columns API strategy. Two construction paths are provided:

- `from_columns(map<string, ChunkedArray>)` when data already exists in Arrow chunked form.
- `from_columns(vector<pair<string, Array>>)` for ordered insertion while wrapping each array into a one-chunk **ChunkedArray**.

This dual API is useful because internal operations often produce flat arrays while external callers may already use chunked data.

Output strategy. CSV writes are performed through `WriteCSV` after `CombineChunksToBatch()`, and Parquet writes use `parquet::arrow::WriteTable`. Both methods throw detailed runtime errors when status objects indicate failure.

1.3 Strategy for Projection, Filtering, and Derived Columns

Projection (`select`). The implementation loops through requested names, resolves both field and column by name, and builds a new schema plus array list. Missing names trigger exceptions immediately. Complexity is approximately linear in number of selected columns and independent of row count, since it mostly reuses existing arrays.

Filtering (filter). The method evaluates an expression to a mask and enforces that the expression’s result type is boolean. A key strategy here is explicit mask application column-by-column:

1. Evaluate predicate to a boolean chunked array.
2. Flatten mask for contiguous row access.
3. For each column, flatten and apply `applyBooleanMask(...)`.
4. Rebuild a new table with identical schema and filtered arrays.

This avoids hidden behavior in third-party filtering APIs and gives full control over null handling and type dispatch in utility functions.

Column creation/replacement (with_column). The expression is evaluated once, then existing column with same name (if present) is skipped during reconstruction. The new column and its field are appended. This realizes a “replace-if-exists, append-if-new” policy.

1.4 Strategy for Head and Sorting

Head. `head(n)` slices each column using `Arrow Slice(0, min(n, length))`. Since slicing is per-column, row count in all output columns remains consistent.

Sorting strategy details. Sorting has been implemented with a precomputation-first approach:

- Sort-key columns are flattened once and converted into in-memory typed vectors.
- Numeric values are promoted to `double` in key storage to unify comparison logic.
- Null masks are stored explicitly per sort key and compared before value comparisons.
- `std::stable_sort` is used to preserve deterministic ordering for ties.
- Post-sort reordering is performed for all columns using a single sorted index list.

This strategy shifts work from the comparator into preprocessing, reducing overhead during $O(N \log N)$ compare calls.

1.5 Join Strategy in Depth

The join implementation is a hash-join with practical engineering choices for performance and correctness:

- 1) **Build side selection.** The smaller relation is chosen as hash build side to reduce memory footprint and improve cache locality.
- 2) **Key encoding.** Composite keys are serialized to compact binary strings. For each key component:

- nulls are marked with a dedicated sentinel byte,
- non-null numeric values are appended as raw bytes,
- strings are appended directly,
- a delimiter byte separates components.

This avoids repeated scalar-to-string conversion for numeric keys.

3) Probe and match collection. Probe rows generate keys and lookup in hash map `key -> vector<row_indices>`.

4) Outer semantics. For `left`, `right`, `outer` joins, unmatched tuples are explicitly appended with `-1` for missing side index.

5) Output schema strategy. Join keys are coalesced into one key column in output. Non-key right columns are appended, and conflicts are disambiguated by `_right` suffix.

1.6 GroupBy and Aggregation Strategy in Depth

Grouping state representation. Grouping first pre-flattens all group-key arrays and numeric candidate aggregate arrays. Group identity is encoded as binary key and aggregated through `unordered_map` for expected $O(1)$ updates.

Accumulator design. For each (group, column), `GroupAccumulator` stores:

$$sum, count, min, max$$

so all supported aggregate outputs can be derived without rescanning data.

Finalization strategy. Aggregation outputs are materialized column-wise, with output Arrow type selected per operation and source type (for example, `count` always as `int64`).

1.7 Complexity Summary for Eager Operations

Operation	Main Strategy	Asymptotic Cost (rough)
<code>select</code>	Schema+array reconstruction from named columns	$O(k)$ for k selected columns
<code>filter</code>	Evaluate mask then apply to each column	$O(R \cdot C)$
<code>with_column</code>	Evaluate expression once and rebuild schema/arrays	$O(R + C)$
<code>sort</code>	Precompute keys + stable sort + reorder columns	$O(R \log R + R \cdot C)$
<code>join</code>	Hash join with smaller-side build	Expected $O(R_l + R_r + M)$

<code>group_by+aggregate</code>	Hash grouping + accumulator finalization	Expected $O(R \cdot G)$
---------------------------------	--	-------------------------

2 LazyDataFrame and Exactly How I Am Making a Graph Using `std::variant`

2.1 Why `std::variant` Was Chosen

The logical plan is represented as a value-type sum over node structs:

```
planNode = variant<ScanNode, FilterNode, SelectNode, WithColumnNode, GroupByNode, AggNode, ...>
```

This gives compile-time exhaustiveness with `std::visit` and removes the need for a base-class hierarchy with virtual dispatch for plan nodes.

2.2 Plan Node Data Model

Every node contains exactly the parameters needed for that relational operator:

- `ScanNode`: file path + CSV/Parquet flag.
- Unary operators: payload + `child` pointer.
- `JoinNode`: `left`, `right`, join keys, join type.
- `HeadNode`: row limit n + child.

The data model is explicit and serializable to DOT labels easily.

2.3 How Lazy Operations Build the DAG

Each chained API call wraps the old root in a new node:

- `scan_csv(path)` starts with `ScanNode(path, is_csv=true)`.
- `filter(pred)` creates `FilterNode(pred, oldRoot)`.
- `select(cols)` creates `SelectNode(cols, oldRoot)`.
- `with_column(name, expr)` creates `WithColumnNode(name, expr, oldRoot)`.
- `group_by(keys).aggregate(aggs)` forms `GroupByNode` then `AggNode`.
- `join(other,...)` creates binary `JoinNode(oldRoot, otherRoot,...)`.

No data is touched during these calls. Only node objects and pointers are constructed.

2.4 From DAG to Execution: Variant-Visitor Runtime

`collect()` follows this pipeline:

1. optimize logical plan,
2. apply projection pushdown pass,
3. execute optimized plan recursively.

Execution uses `std::visit` where each node case maps directly to eager operator calls. This makes semantics transparent: plan nodes are merely delayed forms of eager operations.

2.5 Exactly How Graph Rendering Is Implemented

The graph rendering path is deterministic and two-step:

Step 1: DOT generation.

- `LazyDataFrame::explain(path)` calls optimizer first.
- It opens `path + ".dot"` and writes graph preamble (direction, node style).
- It calls recursive helper `writeDot(out, optimized, nodeId)`.

Step 2: PNG rendering.

- System command: `dot -Tpng <dotfile> -o <path>`.
- Non-zero return value raises runtime error.

Inside `writeDot`:

- A unique integer `myId` is assigned to each visited node.
- `std::visit` pattern-matches node type and prints a readable label:
 - Scan labels include file path,
 - Filter labels include predicate expression string,
 - Select/GroupBy/Agg labels include column/operator payload,
 - Join labels include join type and two child edges,
 - Sort labels include order direction,
 - Head labels include row limit n .
- For child nodes, recursion returns child ids used to write `parent -> child` edges.

So the variant-based DAG is not only used for execution but also for user-facing plan explainability.

2.6 Correctness Intuition for Lazy-to-Eager Mapping

For each node type, execution rule in `QueryExecutor` is exactly the eager counterpart. Therefore, for a fixed plan P , the result of `lazy collect()` equals eager composition of the same operations in the same tree shape (modulo safe optimizer rewrites).

3 QueryOptimizer and Correctness of the Transformations That I Have Made

3.1 Optimizer Architecture

The optimizer has two public static entry points:

- `optimize(planNode input)`: bottom-up rewrites and local transformation rules.
- `pushdownProjections(const planNode& input)`: dependency-driven column pruning pass.

The first pass simplifies expressions and pushes operators where safe. The second pass computes required columns and rewrites plan to avoid unnecessary column flow through the DAG.

3.2 Transformation A: Constant Folding and Algebraic Simplification

What is implemented. The optimizer detects constant-only expressions by checking absence of `col(...)` references in `toString()`. Such expressions are evaluated on a one-row dummy table and replaced with literal expressions. In addition, recursive simplifications are applied:

- $x + 0 = x, 0 + x = x, x - 0 = x$
- $x \cdot 1 = x, 1 \cdot x = x$
- $x \cdot 0 = 0, 0 \cdot x = 0$
- $x/1 = x$
- $true \wedge x = x, false \wedge x = false$
- $false \vee x = x, true \vee x = true$
- $\neg(\neg x) = x$

Correctness argument. All rewrites are tautological identities in arithmetic/boolean algebra. Replacing subtree E by equivalent E' preserves row-wise evaluation for every input tuple. Since relational operators consume only these values, table output remains unchanged.

3.3 Transformation B: Predicate Pushdown

Rules implemented.

- Filter through Select: always safe.
- Filter through Sort: always safe.
- Filter through WithColumn: safe only if predicate does not reference newly created column token.
- Filter through Join: side-aware and join-type-aware conditions.

Join-aware safety used in code.

- **inner**: push to either side if predicate references only that side.
- **left**: push only to right side.
- **right**: push only to left side.
- **outer**: no push.

If predicate columns are all join keys, the code may push to both sides for applicable join kinds.

Correctness argument. For unary row-preserving operators (Select, Sort), predicate commutation is straightforward. For WithColumn, the guard avoids changing dependencies. For joins, restrictions preserve null-extension behavior of outer joins and therefore avoid cardinality/row-content regressions.

3.4 Transformation C: Limit Pushdown

Implemented rules:

`Head(Select(X)) -> Select(Head(X)), Head(WithColumn(X)) -> WithColumn(Head(X))`

No pushdown through operators where semantics may change (sort/group/aggregate/join/filter).

Correctness argument. Both Select and WithColumn preserve row order and count. Therefore, taking first n rows before or after these operators yields identical row set and order.

3.5 Transformation D: Projection Pushdown

The second pass recursively propagates `requiredCols`:

- Filter extends requirements with predicate-referenced columns.
- Sort extends with sort keys.

- WithColumn is removed when produced column is never used.
- Agg trims aggregation list to needed outputs and pushes source requirements.
- Join splits requirements into left/right subsets plus mandatory join keys.

Important practical note from current code. At ScanNode, no physical file-level column pruning is done yet because eager scan currently reads full tables. So this pass reduces intermediate column flow in memory but does not yet reduce scan I/O.

Correctness argument. If a column is removed, it means no downstream operator needs it either directly or indirectly. Dependency closure is preserved by adding all operator-required columns (predicates, sort keys, aggregate inputs, join keys). Therefore, removed columns are observationally irrelevant.

3.6 End-to-End Correctness Statement

Let P be original plan and P' be optimized plan. Each rewrite in the implemented set is semantics-preserving under its explicit guard conditions. Since rewrites are composed sequentially and each step preserves semantics, final plan preserves semantics:

$$\forall D, \text{execute}(P, D) = \text{execute}(P', D)$$

where D is any valid input dataset satisfying schema/type assumptions.

3.7 Performance Impact Summary

Expected gains from implemented transforms:

- Less expression evaluation work via simplification/folding.
- Fewer rows entering expensive operators via predicate pushdown.
- Lower column movement overhead in intermediate nodes via projection pass.
- Earlier cardinality reduction via safe head pushdown.

4 Design Decisions

Certain design decisions made by me:

- Made graph using GraphWiz/Dot
- Applied optimization in explain() as well
- Implemented the lazydataframe with std::variant similar to a functional programming language(kind of inspired from Ocaml's/Typescript's SUM type)

5 Conclusions

This project has a coherent two-engine architecture: an eager execution core over Arrow tables and a lazy query-planning layer over a variant-based DAG. The eager side is implementation-driven and explicit, with stable ordering in sort, hash-based join, and accumulator-based grouping. The lazy side captures query intent and supports explainability through DOT/Graphviz rendering of optimized plans.

The main things that I realized while doing the assignment was a) Initial directory structure and understanding problem statement is crucial and b) Maintaining correctness is very important and needs to be proved rigorously.